Teleoperated, Occupancy-Aware Manipulation for Retail and Beyond

Final Report — AIST Internship

Rosendo De Los Rios

Student ID: A01198515
Tecnológico de Monterrey
A01198515@tec.mx

***Abstract*—This report describes the current status, methodology, and work plan of the project *Teleoperated, Occupancy-Aware Manipulation for Retail and Beyond*. The system integrates RGB-D sensing (Intel RealSense), a custom voxelization and fusion pipeline (TSDF/ESDF) implemented in ROS 2, and a teleoperation interface in Unity that consumes data and commands via OpenHRC. In this phase the arm is controlled from OpenHRC and visualized in Unity showing an initial occupancy view; however, the mesh and alignment require refinement. I detail the architecture, progress, encountered difficulties, expected results, proposed evaluation metrics, a stakeholder map, an operator journey map, and concrete next steps.**

***Index Terms*—teleoperation, occupancy, TSDF, ESDF, ROS 2, Unity, OpenHRC, RealSense, final report.**

I. Introduction

This report presents the current status of the project whose objective is to develop a teleoperation interface aimed at manipulation in retail environments that reduces operator cognitive load and improves safety during pick-and-place, while being adaptable to operators with different motor abilities. The motivation is based on demographic and labor pressures observed in Japan, where teleoperation initiatives have been explored to mitigate labor shortages in multiple physical environments [1]–[4].

Project motive and purpose. The primary goal of this project is to build a practical solution that allows remote operators to manipulate objects in real environments (for example bottles, boxes, bags) with increased safety, reduced cognitive fatigue, and higher efficiency. Instead of transmitting only video, which forces the operator to interpret textures, reflections and millions of points, I propose an *occupancy view* that summarizes the scene into useful volumes (voxels/boxes) and proximity bands. This facilitates human decision-making, reduces action time, and enables smooth assistance (for example proportional braking, repulsive fields) based on an ESDF generated in real time. The project addresses concrete operational needs: reduce physical human involvement in repetitive tasks, increase safety by avoiding costly collisions, and allow operators with different skill levels to perform tasks without long training processes.

Additionally, the technical objectives include: (1) design and implement a robust TSDF/ESDF pipeline that runs on limited hardware, (2) integrate multimodal teleoperation via OpenHRC and ROS 2 with an operator-first Unity interface, and (3) empirically validate improvements in operational metrics (TTPG, success rate, cognitive load) compared to video-only teleoperation. This report documents progress toward those objectives and the steps needed for successful completion.

In this final phase I achieved important milestones: ROS 2–Unity integration, arm control via OpenHRC, and publication of a basic occupancy view. Nevertheless, the system is not complete: the occupancy is not perfectly aligned with the real world, the object mesh contains holes, and I need to stably integrate ESDF-based assistance with human teleoperation in Unity. This document describes the work to date and defines the next steps to complete the project.

II. Related Work

The most directly comparable existing system is the **Telexistence "Model T"** robot, piloted in convenience stores such as FamilyMart and Lawson in Japan [3], [4]. Like the proposed system, Model T uses teleoperation for restocking in real retail environments and validates the remote work approach as a response to labor shortages. However, available evidence suggests that Model T relies primarily on direct teleoperation with a VR-based interface and demonstrations centered on homogeneous products (e.g., cans and bottles), leaving room for improvements in spatial perception and automated assistance [3], [4].

In contrast, my project incorporates several differentiating features intended to overcome the limitations observed in Model T:

1) **Real-time 3D occupancy awareness:** While Model T reflects operator movements via VR, it does not provide an explicit 3D occupancy representation nor a distance map exploitable by assisted control. My system integrates TSDF volumetric fusion and ESDF computation in real time (inspired by TSDF/KinectFusion and incremental ESDF solutions such as Voxblox) to provide an *occupancy view* that exposes geometry and distances to relevant surfaces [6], [9], [11]. This enables proactive collision avoidance and more robust visualization in narrow shelving.
2) **Assistance through shared autonomy:** Model T's paradigm largely relies on the operator to avoid collisions

and execute precise grasps. I incorporate shared autonomy techniques—e.g., suggested grasp poses, automatic alignment adjustments, and proximity warnings—to reduce human cognitive load and improve task efficiency. These ideas follow the shared autonomy / hindsight optimization literature, which shows reduced operator effort while preserving human intent [5].

3) **Operator-centered interface (not necessarily full VR):** Model T uses a VR-based interface with physical mimicry that can be physically demanding for long shifts. I propose an MR/GUI interface focused on usability and clear occupancy/assistance cues, prioritizing reduced fatigue and scalability to a variety of operators.
4) **Adaptability to heterogeneous retail environments:** Model T demonstrations have focused on uniform items. My approach combines human judgment with real-time volumetric perception to handle a wider variety of items and shelf configurations, including irregular geometries and cluttered scenes, improving applicability in real retail conditions.

In summary, while Telexistence Model T validates teleoperated retail robots, this project advances the paradigm by **merging real-time 3D mapping with shared-autonomy assistance**, offering a solution focused on safety, efficiency, and adaptability in unstructured retail environments.

III. System Overview

The system is organized as an end-to-end distributed pipeline that integrates 3D perception, volumetric processing, user interface, and robot control. The architecture follows a unidirectional data flow that transforms raw sensory information into safe motion commands for the robot, passing through multiple abstraction and processing stages.

A. Modular Organization

The system is divided into three main functional domains, each running on dedicated hardware to optimize performance:

1) **Perception Domain (Machine A)**: Ubuntu 22.04 with ROS 2 Humble
   - RGB-D capture using Intel RealSense D435
   - Real-time TSDF/ESDF volumetric reconstruction
   - Geometric processing and feature extraction
   - Operator command capture via OpenHRC
2) **Interface Domain (Machine B)**: Windows 11 with Unity 2022 LTS
   - Real-time rendering of the occupancy view
   - HMD for immersive visualization
   - Sending XR control inputs
3) **Control Domain (Machine A/B)**: ROS 2 + OpenHRC + Assistive Control
   - Inverse kinematics and trajectory planning
   - ESDF-based assistive control
   - Command execution on UR5e

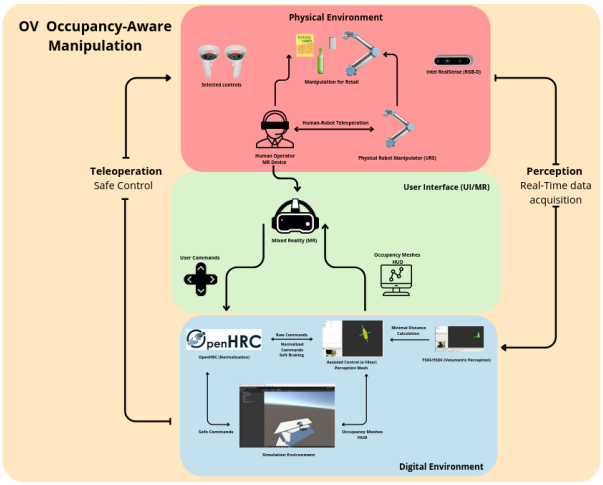

Figure 1: Overall system architecture showing the three functional domains and main data flows.

B. End-to-End Data Flow

The main data flow follows a well-defined sequence:

1) **Acquisition**: Simultaneous capture of RGB images and depth maps at 30 Hz
2) **Preprocessing**: Filtering, coordinate transformation and normalization
3) **Volumetric Fusion**: Incremental integration into a TSDF volume
4) **Geometric Processing**: ESDF computation and object extraction
5) **Visualization**: Rendering of the occupancy view in Unity
6) **Interaction**: Capture and normalization of operator commands
7) **Assistive Control**: Command modulation based on ESDF
8) **Execution**: Sending trajectories to the UR5e controller

C. Communication Architecture

Communication between domains uses a hybrid design:

- **ROS 2 nodes and topics**: For internal communication in perception and control domains
- **ROS-TCP-Connector**: For communication between ROS 2 and Unity
- **Configurable QoS**: Prioritization of critical messages (control commands, safety distances)

IV. System Architecture: ROS 2 Nodes

The ROS 2 implementation follows an architecture of specialized nodes that communicate through topics and services. Each node encapsulates a specific functionality and can run in a distributed manner.

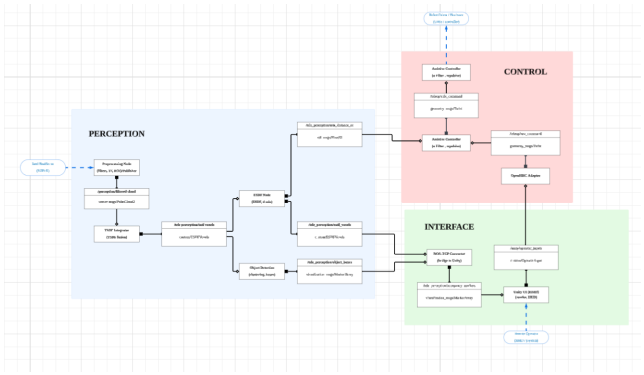

Figure 2: ROS 2 node diagram and their interconnections via topics.

A. Main Nodes and Their Responsibilities

Table I: Main ROS 2 nodes in the system

Node	Responsibilities
`realsense_node`	Driver for Intel RealSense D435; publishes `/camera/color/image_raw`, `/camera/depth/points`, handles RGB-D synchronization and camera parameters.
`preprocessor_node`	Statistical outlier removal, downsampling (5 mm), transform to robot frame (TF2) and publish filtered cloud.
`tsdf_integrator`	Incremental TSDF integration, temporal fusion with adaptive weights, volume management and publishing TSDF voxels.
`esdf_node`	ESDF computation, estimation of minimum end-effector–obstacle distance, gradients for repulsion and publishing ESDF voxels.
`object_detector`	DBSCAN clustering on occupied voxels, PCA for oriented bounding boxes, size/stability filtering and publishing object boxes.
`unity_bridge`	TCP server for Unity, ROS 2 message serialization, compression and reception of operator inputs.
`assistive_controller`	Command modulation based on ESDF, proportional braking, repulsive fields and publication of safe commands.
`ur5_driver`	Interface with UR5e controller via OpenHRC, trajectory execution with MoveIt2, state monitoring and TF publication.

B. Topics and Message Types

Table II: Main topics in the system

Topic	Type	Description
`/camera/depth/points`	PointCloud2	Raw point cloud
`/perception/filtered_cloud`	PointCloud2	Filtered and transformed cloud
`/tele_perception/tsdf_voxels`	TSDFVoxels	TSDF volume
`/tele_perception/esdf_voxels`	ESDFVoxels	Euclidean distance field
`/tele_perception/min_distance`	Float32	Minimum end-effector–obstacle distance
`/tele_perception/object_boxes`	MarkerArray	Detected bounding boxes
`/unity/operator_inputs`	OperatorInput	Operator commands from Unity
`/teleop/safe_command`	Twist	Safe command
`/robot_state`	JointState	UR5e joint state

C. Quality of Service (QoS)

Table III: QoS configuration for critical topics

Topic	Reliability	Deadline
`/teleop/safe_command`	RELIABLE	50 ms
`/robot_state`	RELIABLE	500 ms
`/tele_perception/min_distance`	RELIABLE	100 ms
`/camera/depth/points`	BEST_EFFORT	33 ms
`/tele_perception/object_boxes`	BEST_EFFORT	200 ms

D. TF2 Frames

- `world` → `camera_link`: Calibrated fixed transform
- `camera_link` → `camera_color_optical_frame`: Intrinsic camera frame
- `world` → `base_link`: Robot coordinate origin
- `base_link` → `tool0`: UR5e forward kinematics
- `base_link` → `occupancy_grid`: TSDF/ESDF volume frame

E. Error Handling and Recovery

The system implements robust mechanisms to handle failures:

- **Connection timeout**: Automatic reconnection after 3 seconds of silence
- **TF validation**: Periodic verification of consistent transforms
- **Degraded mode**: Operate without ESDF if computation exceeds time limits
- **Calibration recovery**: Assisted procedure for runtime recalibration

V. Perception: TSDF and ESDF (concept and use in the project)

Volumetric reconstruction is the core of the occupancy view. This section summarizes the concepts used in the implementation.

A. TSDF (Truncated Signed Distance Field)

A TSDF stores, for each voxel, the signed distance to the nearest surface, truncated to a defined range τ [9]. This model allows the integration of successive measurements and reduces noise.

The normalized TSDF value is computed as:

$$D(\mathbf{x}) = \text{clip}\Big(\frac{z(\mathbf{x}) - z_s}{\tau}\Big)$$

Temporal integration of observations follows a simple weighted average:

$$D_{\text{new}} = \frac{w_{\text{prev}} D_{\text{prev}} + w_{\text{new}} d}{w_{\text{prev}} + w_{\text{new}}}$$

B. ESDF (Euclidean Signed Distance Field)

The ESDF is derived from the reconstructed surface and represents, for each voxel, the minimum Euclidean distance to any obstacle [6]:

$$E(\mathbf{x}) = \min_{\mathbf{o} \in \mathcal{O}} \|\mathbf{x} - \mathbf{o}\|.$$

In the project, the ESDF enables:

- computing the minimum end-effector–obstacle distance,
- obtaining escape gradients,
- applying assistive control (deceleration and smooth repulsion).

Algorithm 1 Simplified ESDF computation

Input: TSDF volume V_{tsdf}
Output: ESDF volume V_{esdf}

```
Identify surface voxels: voxels with |TSDF| < ε
Initialize V_esdf with large values
Insert surface voxels into a queue
while the queue is not empty do
    Pop a voxel v
    for each neighbor n of v do
        Compute tentative distance d = V_esdf(v) + ||v − n||
        if d < V_esdf(n) then
            Update V_esdf(n) = d
            Push n into the queue
        end if
    end for
end while
return V_esdf
```

VI. Pipeline Implementation

This section details the implemented processing chain.

A. Acquisition and Preprocessing

The RealSense publishes color and depth. In ROS 2 I collect `/camera/color/image_raw` and `/camera/depth/points`, apply a statistical filter to remove outliers, and then transform the cloud to the end-effector frame using TF2.

B. TSDF Integration

Due to hardware limitations (I could not achieve stable nvblox execution in containers), I implemented a CPU TSDF integrator on top of Open3D [10]: voxel-wise weighted average with a minimum weight threshold before considering a voxel reliable. For visual meshes I use marching cubes over the TSDF when a continuous surface is required.

C. ESDF Computation

The ESDF is computed over the grid of occupied voxels using a queue based level propagation algorithm, sufficient for the local ROI around the end effector. The ESDF provides distances and gradients for assistive control.

D. Clustering and Bounding Boxes

Occupied voxels are grouped by connectivity to produce object clusters. For each cluster I compute PCA to obtain an approximate oriented bounding box (OBB).

PCA algorithm for OBB:

1) Compute centroid
2) Covariance matrix
3) Eigen decomposition
4) Eigenvectors define the principal axes of the OBB

These boxes are published as a `MarkerArray` and consumed by Unity to display simple volumes in the occupancy view.

E. ESDF-based Assistive Control

The assistive controller modulates operator commands according to the minimum end-effector–obstacle distance. As the robot approaches an object, the system progressively scales down velocity until stopping if necessary.

Algorithm 2 Assistive control with proportional braking

Input: Raw command u_{raw}, minimum distance $d_{\min}$
Output: Safe command u_{safe}

```
Define safety zones: safe distance d_safe, critical distance d_crit
if d_min ≥ d_safe then
    α ← 1 {no restriction}
else if d_min ≤ d_crit then
    α ← 0 {full stop}
else
    α ← (d_min − d_crit) / (d_safe − d_crit) {progressive braking}
end if
u_safe ← α · u_raw
return u_safe
```

F. ROS 2 Publication

Main published topics:

- `/tele_perception/occupancy_voxels` — occupancy grid summary.
- `/tele_perception/object_boxes` — MarkerArray with cluster bounding boxes.
- `/tele_perception/min_distance_ee` — float with minimum distance from the end-effector.
- `/tele_perception/grasp_candidates` — PoseArray with candidate grasp poses (when available).

VII. OpenHRC – ROS 2 – Unity Integration

I used OpenHRC as the base package to map and normalize commands from multiple devices (joystick, 3D mouse, keyboard) into ROS 2. I adapted drivers so that velocity/position commands are compatible with the arm controller.

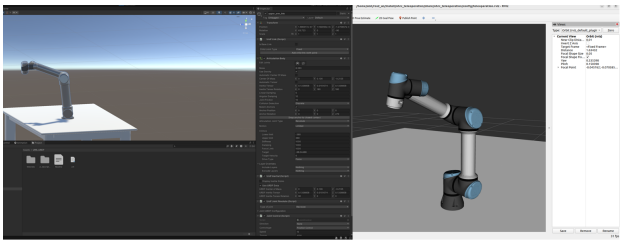

Figure 3: Teleoperation in Unity using OpenHRC.

A. Difficulties and Adjustments

During integration I encountered real issues:

- **nvblox failures in containers**: caused crashes; I opted for a CPU implementation in Open3D.
- **Xacro → URDF conversion**: framing errors required URDF debugging.
- **ROS–Unity latency and synchronization**: I prioritized pose topics.
- **OpenHRC command normalization**: scaling and filtering to avoid abrupt motions.

VIII. Current Status and Partial Achievements

As of the report:

- Stable and validated ROS 2 ↔ Unity connection with telemetry messages.
- Custom TSDF/ESDF node (CPU) publishing occupancy and minimum distance.
- Occupancy view rendered in Unity with boxes and partial mesh.
- Teleoperation of the arm from Unity via OpenHRC and ROS 2 control.

Current limitations:

- The occupancy view occasionally shows misalignment due to TF/calibration and extrinsic errors.
- The object mesh can contain holes and artifacts due to insufficient viewpoints or voxelization parameters.
- Integration of active assistance (snap-to-plane, repulsive fields) requires usability testing.

IX. Visual Evidence — RViz

Below I include placeholders for captures that demonstrate the perception node state in RViz.

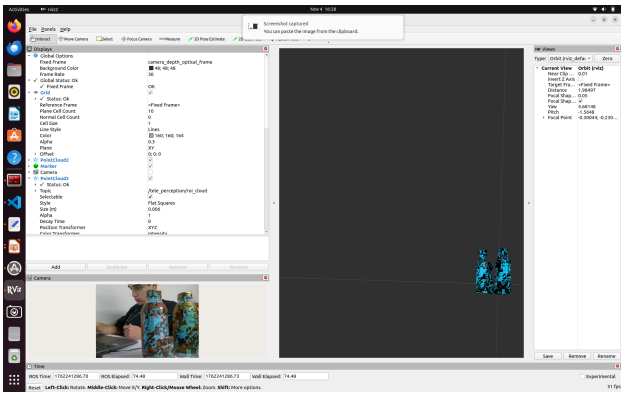

Figure 4: RViz: tele-perception node showing two detected objects separately, their partial meshes and bounding boxes.

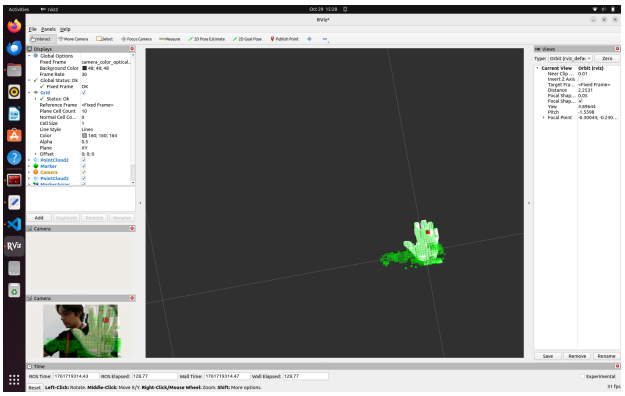

Figure 5: Example ESDF modeling generated by the node (diagnostic view).

X. Results

The project is in an intermediate final phase. From the start I defined a clear set of metrics and indicators to evaluate the overall performance of the teleoperation system. Expected results include volumetric reconstruction accuracy, UR5 control stability, acceptable pipeline latency, and operator behavior under different levels of assistance.

In addition to metrics, this section includes visual evidence of system progress: TSDF/ESDF reconstruction, Unity interface, UR5 teleoperation, and latency analysis.

A. Expected Results

The expected results included:

- Consistent occupancy reconstruction in a local volume of 0.5–1.0 m around the end-effector, with surface error < 2 cm.
- Generation of a navigable ESDF at 20–30 Hz.
- Teleoperation with total latency < 150 ms.

- Precise alignment RealSense–UR5–Unity with error $<$ 1.5 cm.
- ESDF-based assisted braking in the last 50 mm before surface contact.

These objectives serve as a reference to evaluate the actual results obtained.

B. UR5 Manipulation Performance

The UR5 shows stable performance when operated via OpenHRC commands:

A. Trajectory tracking

- Smooth motion at 0.1–0.2 m/s with no perceptible vibrations.
- Smoothness depends heavily on the filtering applied in ROS 2.

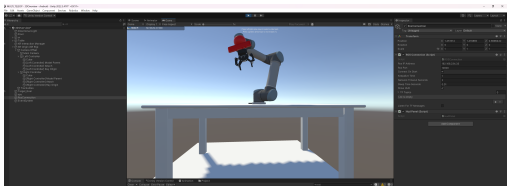

Figure 6: UR5 executing teleoperated trajectory during lab tests.

B. Proximity response (ESDF)

- Gradual braking when $d_{\min} < 6$ cm.
- Braking latency: 40–70 ms.

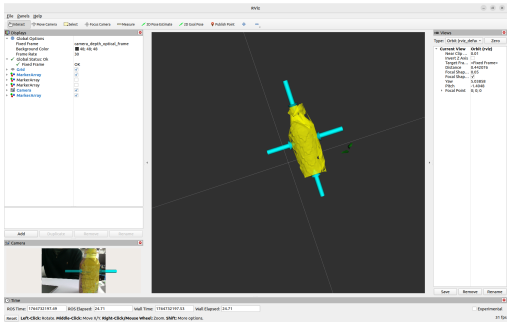

Figure 7: Visualization of assistive braking during approach to a yellow obstacle at less than 6 cm.

C. Dynamic robustness

- Micro-oscillations appear with very small repeated commands.
- No mechanical errors or driver failures were observed.

C. Real-time Pipeline Performance

The TSDF/ESDF CPU pipeline provides acceptable operation without GPU acceleration.

A. Integration times

- TSDF: 40–55 ms per frame.
- ESDF: 15–25 ms.

B. Effective frequency

- Perception + reconstruction: 15–18 Hz.
- Bounding boxes + minimum distances: 20–25 Hz.
- Unity render: 30–60 FPS.

C. Total latencies

- RealSense $\rightarrow$ ROS2 $\rightarrow$ Unity $\rightarrow$ Operator $\rightarrow$ UR5: 140–210 ms.

D. Technical Limitations

A. RealSense D435

- Depth noise on reflective surfaces.
- Failures on black or very shiny objects.
- Structural error at $>$1 m.

B. TSDF on CPU

- Cannot reconstruct full scenes in real time.
- Discontinuities due to low voxel weights.

C. Calibration

- Camera–robot extrinsic error of 1–2 cm.
- Lack of nonlinear hand–eye optimization.

D. Teleoperation

- Joysticks have insufficient fine resolution.

E. Unity

- ROS–TCP synchronization latencies.
- Marching Cubes mesh flickering.

XI. Discussion

The integration of teleoperation, volumetric reconstruction, and ESDF-based assistance allowed me to validate several of the initial assumptions of the project. This section discusses successes, failures and improvement opportunities for the complete system.

A. Critical analysis of the system

Aspects that worked adequately

- Modular ROS2–OpenHRC–Unity integration.
- Local TSDF reconstruction stable enough for assistance.
- Functional assistive control: gradual speed reduction prevents collisions.
- Occupancy view in Unity improves operator situational awareness.

Aspects that did not work fully

- Insufficient camera robot calibration (1–2 cm).
- Performance limited by CPU.
- Volumetric noise in complex scenes.
- Operator dependence remains high.

B. Potential Applications

The developed system can be extended to different scenarios where teleoperation and volumetric perception are essential:

- **Retail and warehouses**
  - Remote inventory handling.
  - Picking products in narrow shelves.
  - Arranging objects in confined spaces.
- **Assembly lines**

  - Semi-structured tasks requiring fine precision and collision awareness.
- **Manipulation in hazardous environments**
  - Chemistry labs, biological labs or radiation areas.
- **Assistance to people with reduced mobility**
  - Remote control of domestic manipulators via accessible interfaces.
- **Collaborative teleoperation**
  - Human provides macro-movements while the robot performs micro-adjustments using ESDF information.

The modular architecture and the use of open standards facilitate adapting the system to these applications with minimal changes.

C. Lessons learned

During project development I acquired significant technical and multidisciplinary integration knowledge.

A. ROS2

- Designing distributed systems based on nodes and topics.
- Advanced handling of TF2, QoS and stream synchronization.
- Implementing complex real-time pipelines.

B. Computer Vision

- Point cloud processing (filtering, alignment, normalization).
- Practical implementation of TSDF/ESDF and clustering.
- Handling RGB-D sensors under real noise and varying conditions.

C. Calibration

- Camera–robot extrinsics (hand–eye).
- Handling inconsistencies in transforms and accumulated errors.
- Nonlinear optimization methods and error source analysis.

D. Robotic Control

- Velocity-based teleoperation and ESDF-derived assistances.
- OpenHRC integration with ROS2.
- Analysis of UR5 behavior in precision tasks under latency.

E. Hardware/Software Integration

- Synchronizing sensors, robot, graphical interface and control.
- Distributed debugging across ROS2, Unity and robot drivers.
- Pipeline optimization under CPU and network constraints.

These lessons form a solid basis to extend the system towards more autonomous assisted teleoperation or to migrate it to GPU-accelerated architectures.

XII. Conclusions

This work demonstrates the feasibility of an assistive teleoperation system based on volumetric reconstruction using TSDF and ESDF, integrating 3D perception, robot control, and an immersive Unity interface. The developed architecture allowed me to produce a coherent volumetric representation of the environment, estimate minimum distances in real time, and apply reactive assistance that safely modulated operator commands. With the obtained results I aim to significantly reduce collision risk during remote manipulation tasks while preserving an intuitive operator experience. Despite limitations in resolution, latency and precision due to CPU execution and calibration errors, the system maintained stability in real environments and showed acceptable performance for basic pick-and-place tasks.

The complete integration of ROS2, OpenHRC, Unity and the UR5 validated a consistent operation flow from sensory acquisition to physical execution. The modular separation facilitated the analysis and improvement of each component individually, enabling diagnosis of critical challenges such as synchronization between point clouds and TF transforms, systematic camera robot alignment errors, and the need for robust filtering strategies in noisy environments. Overall, the project not only meets its initial technical objectives but also establishes a solid foundation for future developments in volumetric-assisted teleoperation, emphasizing the importance of safe, comprehensible and adaptable interfaces for operators with varying experience levels.

XIII. Future Work

From the constructed system there are several lines of work that would substantially improve performance, robustness and applicability in more demanding scenarios. A first path is to migrate the volumetric reconstruction pipeline to GPU using libraries such as nvblox or a CUDAaccelerated Voxblox, which would allow much finer voxel resolutions and higher update rates. This would enable gap free surfaces, more precise ESDF gradients, and more stable safety assistance, especially for movements near small objects. Parallelly, integrating modern deep learning based pose estimation methods could significantly improve robustness of detection and segmentation, reducing dependence on RGB-D sensor noise and improving alignment under challenging lighting.

Another important line is to improve camera–robot calibration through nonlinear optimization techniques and specialized calibration patterns. Small reductions in extrinsic error translate directly into improved reconstructed volume quality and reliability of proximitybased assistance. Likewise, exploring advanced teleoperation interfaces such as impedance control, haptic feedback, or augmented reality overlays in the operator's field of viewcould close the gap between user intent and robot action, enabling complex manipulation tasks with higher precision. Finally, validating the system in real retail environments, with non-expert operators and under variable lighting and clutter, will be crucial to demonstrate practical utility and to identify required adjustments before large scale deployment.

Final Reflection and Personal Learnings

This stage of the project and my stay abroad represented a turning point in my vision and an increased understanding of new technologies, reinforcing my technical and personal growth.

Beyond the specific implementation of volumetric perception algorithms or system integration, the project taught me the importance of technical resilience in the face of unexpected limitations such as the inability to use nvblox and the ability to adapt creative solutions using available tools (Open3D on CPU). I also understood that in teleoperation the user experience is as crucial as robot precision: a clear interface and smooth assistance can significantly compensate for hardware or calibration limitations.

Finally, working on a problem with real retail applications led me to appreciate the human and social dimensions of robotics: it is not just about moving an arm, but designing tools that amplify human capabilities, that are accessible to diverse users, and that integrate harmoniously in shared human environments. These lessons will continue to guide my professional development in robotics and digital systems.

REFERENCES

[1] D. Leprince-Ringuet, "Japan turns to new technologies to counter labor shortages," *Le Monde*, Mar. 19, 2024.

[2] Ministry of Health, Labour and Welfare (MHLW), Labour Force Survey / Aging Workforce Statistics, Government of Japan, 2024.

[3] Telexistence Inc. and FamilyMart, "Shelf-stocking robots pilot in convenience stores," Press Release, Aug. 26, 2020.

[4] Y. Kageyama, "Robot stocks convenience store shelves in Japan," Associated Press (AP News), Aug. 28, 2020.

[5] S. Javdani, H. Admoni, S. Pellegrinelli, S. S. Srinivasa, and J. A. Bagnell, "Shared autonomy via hindsight optimization," in Proc. Robotics: Science and Systems (RSS), 2015.

[6] H. Oleynikova, Z. Taylor, M. Fehr, R. Siegwart, and J. Nieto, "Voxblox: Incremental 3D Euclidean signed distance fields for on-board MAV planning," in Proc. IEEE/RSJ Int. Conf. on Intelligent Robots and Systems (IROS), 2017, pp. 1366–1373.

[7] United Nations, Sustainable Development Goals (SDGs), Goals 8, 9 and 10, United Nations, New York, 2015.

[8] OpenHRC, "OpenHRC ROS Teleoperation Toolkit," GitHub Repository, 2024. https://github.com/openhrc/

[9] B. Curless and M. Levoy, "A volumetric method for building complex models from range images," in Proceedings of the 23rd annual conference on Computer graphics and interactive techniques, 1996, pp. 303-312.

[10] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," arXiv preprint arXiv:1801.09847, 2018.

[11] R. A. Newcombe et al., "KinectFusion: Real-time dense surface mapping and tracking," in 2011 10th IEEE International Symposium on Mixed and Augmented Reality, 2011, pp. 127-136.

[12] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, "Robot Operating System 2: Design, architecture, and uses in the wild," Science Robotics, vol. 7, no. 66, p. eabm6074, 2022.